

Assignment 3 prompt

Prompt

generate C++ code using objects, not functions, to implement the following requirements

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A

priority queue is a queue where emails can shift towards the front of the queue

based on a priority status.

- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.
- Test File format:

Command Description

EMAIL <sender category>,

<subject line>,

<date>

The emails in the CEO's Inbox should be placed in queue

based on their sender category and date. The sender

categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next

- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin

with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read

The formatting on that should be:

There are 6 emails to read./n/nNext email:/n/tSender: Boss/n/tSubject: Never Mind/n/tDate: 01-03-2025/n/nThere are 4 emails to read

Claude

Claude Sonnet 5 accessed in web browser via free plan on laptop

```
/*
* Name of Program:      EECS 348 Assignment 3
* Description:          C++ program that prioritizes a busy CEO's email inbox
*                        using a hand-written, list-based MaxHeap that
*                        implements a priority queue. Emails are queued by
*                        sender category, read in this order: Boss,
*                        Subordinate, Peer, ImportantPerson, OtherPerson.
*                        Within the same category, the newest email (by
*                        MM-DD-YYYY date) is read before older ones.
*                        Supports EMAIL, NEXT, READ, and COUNT commands.
* Input:                A test file (given as argv[1]) or stdin containing
*                        EMAIL commands (<sender category>,<subject line>,
*                        <date>) followed by any mix of NEXT, READ, and
*                        COUNT commands, one command per line.
* Output:               Terminal output: how many emails are left to read,
*                        the next email's sender/subject/date on NEXT, and
*                        a warning for any malformed EMAIL line.
* Collaborators:        Claude
* Other Sources:         N/A
*
*
* Author:                Claude
* Creation Date:         10/1/2026
* Revision Date:         10/1/2026
* Revisions:             N/A
*
```

```

*/

#include <iostream>

#include <fstream>

#include <sstream>

#include <vector>

#include <map>

using namespace std;

struct Email {

    string sender, subject, date; // date is MM-DD-YYYY

    int key() const { // bigger key = read sooner

        static map<string, int> rank = {"Boss", 5}, {"Subordinate", 4},
{"Peer", 3},

{"ImportantPerson", 2},
{"OtherPerson", 1}};

        int yyyyymmdd = stoi(date.substr(6)) * 10000 + stoi(date.substr(0, 2))
* 100 + stoi(date.substr(3, 2));

        return rank[sender] * 1000000000 + yyyyymmdd; // category first, then
newest date

    }

};

class MaxHeap { // list (array) based binary max-heap

    vector<Email> h;

public:

```

```

int size() const { return h.size(); }

const Email& top() const { return h[0]; }

void push(const Email& e) {

    h.push_back(e);

    for (size_t i = h.size() - 1; i > 0 && h[i].key() > h[(i - 1) /
2].key(); i = (i - 1) / 2)

        swap(h[i], h[(i - 1) / 2]);

}

void pop() {

    h[0] = h.back();

    h.pop_back();

    for (size_t i = 0, c; (c = 2 * i + 1) < h.size(); i = c) {

        if (c + 1 < h.size() && h[c + 1].key() > h[c].key()) c++;

        if (h[c].key() <= h[i].key()) break;

        swap(h[i], h[c]);

    }

}

};

class Inbox {

    MaxHeap q;

    bool started = false;

    void print(const string& s) { cout << (started ? "\n" : "") << s; started
= true; }

```

```

public:

    void run(istream& in) {

        string line, cmd;

        while (getline(in, line)) {

            if (!line.empty() && line.back() == '\r') line.pop_back();

            istringstream ss(line);

            ss >> cmd;

            ss.ignore();

            if (cmd == "EMAIL") {

                Email e;

                getline(ss, e.sender, ',');

                getline(ss, e.subject, ',');

                getline(ss, e.date);

                q.push(e);

            } else if (cmd == "COUNT") {

                print("There are " + to_string(q.size()) + " emails to
read.\n");

            } else if (cmd == "NEXT" && q.size()) {

                print("Next email:\n\tSender: " + q.top().sender +
"\n\tSubject: " + q.top().subject +

                    "\n\tDate: " + q.top().date + "\n");

            } else if (cmd == "READ" && q.size()) {

                q.pop();

            }

        }
    }

```

```

        }

    }

};

int main(int argc, char** argv) {

    ifstream file(argc > 1 ? argv[1] : "");

    Inbox ceo;

    ceo.run(file ? static_cast<istream&>(file) : cin);

}

```

Code Analysis

```

/*
 * Name of Program:      EECS 348 Assignment 3
 * Description:          C++ program that prioritizes a busy CEO's email inbox
 *                        using a hand-written, list-based MaxHeap that
 *                        implements a priority queue. Emails are queued by
 *                        sender category, read in this order: Boss,
 *                        Subordinate, Peer, ImportantPerson, OtherPerson.
 *                        Within the same category, the newest email (by
 *                        MM-DD-YYYY date) is read before older ones.
 *                        Supports EMAIL, NEXT, READ, and COUNT commands.
 * Input:                A test file (given as argv[1]) or stdin containing
 *                        EMAIL commands (<sender category>,<subject line>,
 *                        <date>) followed by any mix of NEXT, READ, and
 *                        COUNT commands, one command per line.
 * Output:               Terminal output: how many emails are left to read,
 *                        the next email's sender/subject/date on NEXT, and
 *                        a warning for any malformed EMAIL line.
 * Collaborators:        Claude, Connor Wiley
 * Other Sources:        N/A
 *
 *
 * Author:               Claude
 * Creation Date:        10/1/2026

```

* Revision Date: 10/1/2026

* Revisions: N/A

*

*/

```
#include <iostream> // for cin, cout, and basic terminal input/output
```

```
#include <fstream> // for ifstream, used to open the file passed in on the  
command line
```

```
#include <sstream> // for istringstream, used to pick apart each line
```

```
#include <vector> // for vector, used as the backing array for the heap
```

```
#include <map> // for map, used to look up a category's priority number
```

```
using namespace std; // so we don't have to type std:: in front of everything
```

```
struct Email { // struct for a single Email
```

```
    string sender, subject, date; // instantiates sender for the person  
    sending, subject for the topic, and date for the year-month-day they sent
```

```
    int key() const { // instantiates a const for key as an int
```

```
        static map<string, int> rank = {{"Boss", 5}, {"Subordinate", 4},  
{"Peer", 3},
```

```
        {"ImportantPerson", 2},  
{"OtherPerson", 1}}; // maps each category name to its priority number
```

```
        int yyyymmdd = stoi(date.substr(6)) * 10000 + stoi(date.substr(0, 2))  
* 100 + stoi(date.substr(3, 2)); // creates the date into YYYYMMDD as a plain  
int
```

```
        return rank[sender] * 1000000000 + yyyymmdd; // returns the category  
first, then newest date as the value
```

```
    }
```

```
};
```

```

class MaxHeap { // creates the class MaxHeap as an array based binary

    vector<Email> h; // the actual heap storage, a vector instead of a raw
array so it resizes itself

public: // everything below here is public

    int size() const { return h.size(); } // returns how many emails are in
the heap

    const Email& top() const { return h[0]; } // looks at the top email, O(1),
but doesn't check if the heap is empty first

    void push(const Email& e) { // adds an email to the heap

        // O(log n) swaps, same reasoning as before, a heap is a complete
binary tree so its height

        // is log n. But because key() isn't cached, each comparison in this
loop re-does the map

        // lookup and the date parsing from scratch, so the real world cost
per comparison is higher

        // than just comparing two ints like the earlier versions did

        h.push_back(e); // sticks the new email on the end of the vector

        for (size_t i = h.size() - 1; i > 0 && h[i].key() > h[(i - 1) /
2].key(); i = (i - 1) / 2) // keeps comparing to the parent while we outrank
it

            swap(h[i], h[(i - 1) / 2]); // swaps up since we outrank the
parent

    } // ends push function

    void pop() { // removes the top email from the heap

        // O(log n) for the sift down loop, same deal with key() getting
recomputed a lot. Also

        // this function doesn't check if h is empty before touching h[0] and
h.back(), so its on

        // whoever calls pop() to make sure the heap isn't empty first, the

```

```
class itself doesn't guard it
```

```
    h[0] = h.back(); // moves the last email into the root
```

```
    h.pop_back(); // shrinks the vector by one
```

```
    for (size_t i = 0, c; (c = 2 * i + 1) < h.size(); i = c) { // walks  
down from the root while theres at least a left child
```

```
        if (c + 1 < h.size() && h[c + 1].key() > h[c].key()) c++; //  
checks if the right child outranks the left, if so use the right child instead
```

```
        if (h[c].key() <= h[i].key()) break; // if neither child outranks  
us, the heap is good, stop here
```

```
        swap(h[i], h[c]); // swaps down to whichever child won
```

```
    } // ends for loop
```

```
    } // ends pop function
```

```
}; // ends MaxHeap class
```

```
class Inbox { // class that wraps the heap and handles reading commands
```

```
    MaxHeap q; // the heap that holds all the CEO's emails
```

```
    bool started = false; // tracks whether weve printed anything yet, used to  
space out the output
```

```
    void print(const string& s) { cout << (started ? "\n" : "") << s; started  
= true; } // prints a blank line before every message except the first one
```

```
public: // everything below here is public
```

```
    void run(istream& in) { // reads commands from a stream and dispatches  
them
```

```
        // Zooming out, EMAIL and READ are each  $O(\log n)$  because of the heap  
operations above, and
```

```
        // COUNT and NEXT are  $O(1)$ . So for  $m$  total lines in the input this  
whole thing runs in about
```

```

    // O(m log n) time overall, same ballpark as the other versions, just
    with a bigger constant

    // factor per comparison because of how key() works

    string line, cmd; // holds each line, and the first word of that line

    while (getline(in, line)) { // reads lines until the input runs out

        if (!line.empty() && line.back() == '\r') line.pop_back(); //
strips a trailing carriage return if theres one

        istringstream ss(line); // builds a stream out of the line so we
can pick it apart

        ss >> cmd; // reads the first word of the line as the command

        ss.ignore(); // skips the single space right after the command
word

        if (cmd == "EMAIL") { // checks if the command is EMAIL

            Email e; // new email to fill in

            getline(ss, e.sender, ','); // grabs everything up to the
first comma as the sender

            getline(ss, e.subject, ','); // grabs everything up to the
second comma as the subject

            getline(ss, e.date); // grabs whatever is left as the date

            q.push(e); // pushes the new email onto the heap

        } else if (cmd == "COUNT") { // checks if the command is COUNT

            print("There are " + to_string(q.size()) + " emails to
read.\n"); // prints how many emails are left

        } else if (cmd == "NEXT" && q.size()) { // checks if the command
is NEXT and theres actually something to show

            print("Next email:\n\tSender: " + q.top().sender +
"\n\tSubject: " + q.top().subject +

                "\n\tDate: " + q.top().date + "\n"); // prints out the

```

sender, subject, and date of the top email

```
        } else if (cmd == "READ" && q.size()) { // checks if the command  
is READ and theres actually something to remove
```

```
            q.pop(); // pops the top email off the heap
```

```
        } // ends else if statement
```

```
    } // ends while statement
```

```
    } // ends run function
```

```
}; // ends Inbox class
```

```
int main(int argc, char** argv) { // instantiates the main function for  
running the program
```

```
    ifstream file(argc > 1 ? argv[1] : ""); // tries to open a filename if one  
was given, otherwise tries to open an empty name which just fails
```

```
    Inbox ceo; // creates the inbox that holds the heap and does all the work
```

```
    ceo.run(file ? static_cast<istream&>(file) : cin); // uses the file if it  
actually opened okay, otherwise falls back to stdin
```

```
    } // ends main function
```

Improved Code

I Chose Claude for my improvements.

My improvements are:

In terms of being correct top and pop previously just grabbed h[0] and h.back() without checking if the heap was empty so I make it so they run h.empty before attempting those calls. Second, I fixed an issue with key() where it would auto insert a new entry at 0 if you attempted to look up an index that didnt exist, now it just returns the default.

Time Complexation

- In terms of time, key() used to redo the map lookup but not it just caches once and returns that int, stopping the repetition.

- The heap operations are still $O(\log n)$ either way since that's just how tall the heap is, but the comparison is a lot smaller now that it's just comparing two ints instead of redoing all that parsing.

Space Complexation

- In terms of space, caching the key costs one extra int per Email, so that's basically nothing extra per email.
- Getting rid of the map also means I'm no longer carrying around a whole map object anymore, so this version uses less memory, not more.

Maintainability

- In terms of maintainability, the if-chain is a lot easier to read at a glance than digging through a map list.

```

/*
 * Name of Program:      EECS 348 Assignment 3
 * Description:          C++ program that prioritizes a busy CEO's email inbox
 *                        using a hand-written, list-based MaxHeap that
 *                        implements a priority queue. Emails are queued by
 *                        sender category, read in this order: Boss,
 *                        Subordinate, Peer, ImportantPerson, OtherPerson.
 *                        Within the same category, the newest email (by
 *                        MM-DD-YYYY date) is read before older ones.
 *                        Supports EMAIL, NEXT, READ, and COUNT commands.
 * Input:                A test file (given as argv[1]) or stdin containing
 *                        EMAIL commands (<sender category>,<subject line>,
 *                        <date>) followed by any mix of NEXT, READ, and
 *                        COUNT commands, one command per line.
 * Output:               Terminal output: how many emails are left to read,
 *                        the next email's sender/subject/date on NEXT, and
 *                        a warning for any malformed EMAIL line.
 * Collaborators:        Claude, Connor Wiley
 * Other Sources:        N/A
 *
 *
 * Author:               Claude, Connor Wiley
 * Creation Date:         10/1/2026
 * Revision Date:        10/1/2026
 * Revisions:            N/A
 *
 */

#include <iostream> // for cin, cout, and basic terminal input/output

```

```

#include <fstream> // for ifstream, used to open the file passed in on the
command line
#include <sstream> // for istringstream, used to pick apart each line
#include <vector> // for vector, used as the backing array for the heap
using namespace std; // so we don't have to type std:: in front of everything

struct Email { // creates the struct Email for a single email
    string sender, subject, date; // instantiates sender for the person
    sending, subject for the topic, and date for the year-month-day they sent
    int priorityKey = 0; // instantiates a cached key so key() doesn't have to
    redo the lookup and parsing on every single heap comparison

    static int categoryPriority(const string& category) { // instantiates
    categoryPriority as a plain if-chain instead of a map, O(1) since its at most
    4 comparisons, and it cant silently insert a bad entry the way map's
    operator[] used to
        if (category == "Boss") return 5; // Boss is the highest priority
        if (category == "Subordinate") return 4; // Subordinate is next
        if (category == "Peer") return 3; // then Peer
        if (category == "ImportantPerson") return 2; // then ImportantPerson
        return 1; // OtherPerson or anything unrecognized safely falls back to
    the lowest priority, with no side effects
    } // ends categoryPriority function

    void computeKey() { // instantiates computeKey to fill in priorityKey
    once, right after sender and date get set, O(1) since the date is a fixed
    length
        int yyyymmdd = stoi(date.substr(6)) * 10000 + stoi(date.substr(0, 2))
    * 100 + stoi(date.substr(3, 2)); // creates the date into YYYYMMDD as a plain
    int
        priorityKey = categoryPriority(sender) * 100000000 + yyyymmdd; //
    combines the category first, then newest date as the value
    } // ends computeKey function

    int key() const { return priorityKey; } // creates the key function that
    just returns the cached value, O(1), instead of redoing the lookup and parsing
    every time
}; // ends Email struct

class MaxHeap { // creates the class MaxHeap as an array based binary max-
heap
    vector<Email> h; // instantiates h as a vector instead of a raw array so
    it resizes itself
public: // everything below here is public
    int size() const { return h.size(); } // returns how many emails are in
    the heap, O(1)

```

```

    const Email& top() const { // creates the top function that looks at the
top email without removing it, O(1) since its always at the root
        static Email empty; // instantiates a fallback empty email for the
edge case where somebody calls top() on an empty heap
        if (h.empty()) return empty; // checks if the heap is actually empty
first now, instead of just grabbing h[0] no matter what
        return h[0]; // returns the top email
    } // ends top function
    void push(const Email& e) { // creates the push function that adds an
email to the heap, O(log n) since key() is cached now, each comparison during
the sift is just a cheap int compare instead of redoing the lookup
        h.push_back(e); // sticks the new email on the end of the vector
        for (size_t i = h.size() - 1; i > 0 && h[i].key() > h[(i - 1) /
2].key(); i = (i - 1) / 2) // keeps comparing to the parent while we outrank
it
            swap(h[i], h[(i - 1) / 2]); // swaps up since we outrank the
parent
    } // ends push function
    void pop() { // creates the pop function that removes the top email from
the heap, O(log n) for the sift down loop
        if (h.empty()) return; // checks if the heap is actually empty first
now, instead of touching h[0] and h.back() no matter what
        h[0] = h.back(); // moves the last email into the root
        h.pop_back(); // shrinks the vector by one
        for (size_t i = 0, c; (c = 2 * i + 1) < h.size(); i = c) { // walks
down from the root while theres at least a left child
            if (c + 1 < h.size() && h[c + 1].key() > h[c].key()) c++; //
checks if the right child outranks the left, if so use the right child instead
            if (h[c].key() <= h[i].key()) break; // if neither child outranks
us, the heap is good, stop here
            swap(h[i], h[c]); // swaps down to whichever child won
        } // ends for loop
    } // ends pop function
}; // ends MaxHeap class

```

```

class Inbox { // creates the class Inbox that wraps the heap and handles
reading commands
    MaxHeap q; // instantiates q as the heap that holds all the CEO's emails
    bool started = false; // instantiates started to track whether weve
printed anything yet, used to space out the output
    void print(const string& s) { cout << (started ? "\n" : "") << s; started
= true; } // prints a blank line before every message except the first one
public: // everything below here is public
    void run(istream& in) { // creates the run function that reads commands
from a stream and dispatches them, O(m log n) overall for m lines since EMAIL
and READ are each O(log n) and COUNT and NEXT are O(1)

```

```

        string line, cmd; // holds each line, and the first word of that line
        while (getline(in, line)) { // reads lines until the input runs out
            if (!line.empty() && line.back() == '\r') line.pop_back(); //
strips a trailing carriage return if theres one
            istringstream ss(line); // builds a stream out of the line so we
can pick it apart
            ss >> cmd; // reads the first word of the line as the command
            ss.ignore(); // skips the single space right after the command
word
            if (cmd == "EMAIL") { // checks if the command is EMAIL
                Email e; // new email to fill in
                getline(ss, e.sender, ','); // grabs everything up to the
first comma as the sender
                getline(ss, e.subject, ','); // grabs everything up to the
second comma as the subject
                getline(ss, e.date); // grabs whatever is left as the date
                e.computeKey(); // fills in the cached priority key now that
sender and date are set
                q.push(e); // pushes the new email onto the heap
            } else if (cmd == "COUNT") { // checks if the command is COUNT
                print("There are " + to_string(q.size()) + " emails to
read.\n"); // prints how many emails are left
            } else if (cmd == "NEXT" && q.size()) { // checks if the command
is NEXT and theres actually something to show
                print("Next email:\n\tSender: " + q.top().sender +
"\n\tSubject: " + q.top().subject +
"\n\tDate: " + q.top().date + "\n"); // prints out the
sender, subject, and date of the top email
            } else if (cmd == "READ" && q.size()) { // checks if the command
is READ and theres actually something to remove
                q.pop(); // pops the top email off the heap
            } // ends else if statement
        } // ends while statement
    } // ends run function
}; // ends Inbox class

int main(int argc, char** argv) { // instantiates the main function for
running the program
    ifstream file(argc > 1 ? argv[1] : ""); // tries to open a filename if one
was given, otherwise tries to open an empty name which just fails
    Inbox ceo; // creates the inbox that holds the heap and does all the work
    ceo.run(file ? static_cast<istream&>(file) : cin); // uses the file if it
actually opened okay, otherwise falls back to stdin
} // ends main function

```

Gemini

Gemini 3.6 Flash accessed in web browser via free plan on laptop

```
/*
 * Name of Program:      EECS 348 Assignment 3
 * Description:          C++ program that prioritizes a busy CEO's email inbox
 *                       using a hand-written, list-based MaxHeap that
 *                       implements a priority queue. Emails are queued by
 *                       sender category, read in this order: Boss,
 *                       Subordinate, Peer, ImportantPerson, OtherPerson.
 *                       Within the same category, the newest email (by
 *                       MM-DD-YYYY date) is read before older ones.
 *                       Supports EMAIL, NEXT, READ, and COUNT commands.
 * Input:                A test file (given as argv[1]) or stdin containing
 *                       EMAIL commands (<sender category>,<subject line>,
 *                       <date>) followed by any mix of NEXT, READ, and
 *                       COUNT commands, one command per line.
 * Output:               Terminal output: how many emails are left to read,
 *                       the next email's sender/subject/date on NEXT, and
 *                       a warning for any malformed EMAIL line.
 * Collaborators:        Gemini
 * Other Sources:        N/A
 *
 *
 * Author:               Gemini
 * Creation Date:         10/1/2026
 * Revision Date:         10/1/2026
 * Revisions:            N/A
 */

#include <iostream>
#include <string>
#include <vector>
#include <sstream>

using namespace std;

struct Email {
    string sender, subject, date;

    int rank() const {
        if (sender == "Boss") return 5;
        if (sender == "Subordinate") return 4;
        if (sender == "Peer") return 3;
```

```

        if (sender == "ImportantPerson") return 2;
        return 1;
    }

    // Convert MM-DD-YYYY to YYYYMMDD for lexicographical chronological
    comparison
    string dateKey() const {
        return date.substr(6, 4) + date.substr(0, 2) + date.substr(3, 2);
    }

    bool operator<(const Email& other) const {
        if (rank() != other.rank()) return rank() < other.rank();
        return dateKey() < other.dateKey();
    }
};

class MaxHeap {
    vector<Email> heap;

    void siftUp(int i) {
        while (i > 0 && heap[(i - 1) / 2] < heap[i]) {
            swap(heap[(i - 1) / 2], heap[i]);
            i = (i - 1) / 2;
        }
    }

    void siftDown(int i) {
        int size = heap.size(), maxIdx = i, left = 2 * i + 1, right = 2 * i +
2;
        if (left < size && heap[maxIdx] < heap[left]) maxIdx = left;
        if (right < size && heap[maxIdx] < heap[right]) maxIdx = right;
        if (maxIdx != i) {
            swap(heap[i], heap[maxIdx]);
            siftDown(maxIdx);
        }
    }

public:
    void push(const Email& email) { heap.push_back(email); siftUp(heap.size()
- 1); }

    void pop() {
        if (heap.empty()) return;
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) siftDown(0);
    }
};

```

```

    }

    const Email& top() const { return heap[0]; }
    bool empty() const { return heap.empty(); }
    int size() const { return heap.size(); }
};

class CEOEmailManager {
    MaxHeap inbox;

public:
    void processCommands(istream& input) {
        string line;
        while (getline(input, line)) {
            if (!line.empty() && line.back() == '\r') line.pop_back();
            if (line.empty()) continue;

            if (line.rfind("EMAIL ", 0) == 0) {
                stringstream ss(line.substr(6));
                string sender, subject, date;
                getline(ss, sender, ',');
                getline(ss, subject, ',');
                getline(ss, date, ',');
                inbox.push({sender, subject, date});
            } else if (line == "NEXT") {
                if (!inbox.empty()) {
                    const Email& e = inbox.top();
                    cout << "Next email:\n\tSender: " << e.sender
                        << "\n\tSubject: " << e.subject
                        << "\n\tDate: " << e.date << "\n\n";
                }
            } else if (line == "READ") {
                inbox.pop();
            } else if (line == "COUNT") {
                cout << "There are " << inbox.size() << " emails to
read.\n\n";
            }
        }
    }
};

int main() {
    CEOEmailManager manager;
    manager.processCommands(cin);
    return 0;
}

```

```
}
```

Code Analysis

```
/*
 * Name of Program:      EECS 348 Assignment 3
 * Description:          C++ program that prioritizes a busy CEO's email inbox
 *                       using a hand-written, list-based MaxHeap that
 *                       implements a priority queue. Emails are queued by
 *                       sender category, read in this order: Boss,
 *                       Subordinate, Peer, ImportantPerson, OtherPerson.
 *                       Within the same category, the newest email (by
 *                       MM-DD-YYYY date) is read before older ones.
 *                       Supports EMAIL, NEXT, READ, and COUNT commands.
 * Input:                A test file (given as argv[1]) or stdin containing
 *                       EMAIL commands (<sender category>,<subject line>,
 *                       <date>) followed by any mix of NEXT, READ, and
 *                       COUNT commands, one command per line.
 * Output:               Terminal output: how many emails are left to read,
 *                       the next email's sender/subject/date on NEXT, and
 *                       a warning for any malformed EMAIL line.
 * Collaborators:        Gemini, Connor Wiley
 * Other Sources:        N/A
 *
 *
 * Author:               Gemini
 * Creation Date:         10/1/2026
 * Revision Date:         10/1/2026
 * Revisions:            N/A
 */

#include <iostream> // for cin, cout, and basic terminal input/output
#include <string> // for the string type, used for sender/subject/date
#include <vector> // for vector, used as the backing array for the heap
#include <sstream> // for stringstream, used to split EMAIL lines on commas

using namespace std; // so we don't have to type std:: in front of everything

struct Email { // creates the struct Email for a single email
    string sender, subject, date; // instantiates sender for the person
    sending, subject for the topic, and date for the year-month-day they sent

    int rank() const { // instantiates a const function rank that returns an
```

```

int, 0(1) since its just a few comparisons
    if (sender == "Boss") return 5; // Boss is the highest priority
    if (sender == "Subordinate") return 4; // Subordinate is next
    if (sender == "Peer") return 3; // then Peer
    if (sender == "ImportantPerson") return 2; // then ImportantPerson
    return 1; // anything else (including OtherPerson) falls back to the
lowest priority
    } // ends rank function

    // Convert MM-DD-YYYY to YYYYMMDD for lexicographical chronological
comparison
    string dateKey() const { // instantiates a const function dateKey that
returns a string, 0(1) since the date is always a fixed length
        return date.substr(6, 4) + date.substr(0, 2) + date.substr(3, 2); //
creates the date into YYYYMMDD order as a string
    } // ends dateKey function

    bool operator<(const Email& other) const { // creates the operator< so the
heap knows how to compare two emails, 0(1) since it just calls the two
functions above
        if (rank() != other.rank()) return rank() < other.rank(); // checks if
the ranks are different, lower rank means less priority
        return dateKey() < other.dateKey(); // if ranks tie, whoever has the
smaller YYYYMMDD string is older, so less priority
    } // ends operator< function
}; // ends Email struct

class MaxHeap { // creates the class MaxHeap for a list based binary max-heap
    vector<Email> heap; // instantiates heap as a vector instead of a raw
array so it resizes itself

    void siftUp(int i) { // creates the siftUp function that bubbles an email
up until the heap is happy, 0(log n) since the heap is only log n levels tall
        while (i > 0 && heap[(i - 1) / 2] < heap[i]) { // keep going while
were not at the root and the parent is weaker
            swap(heap[(i - 1) / 2], heap[i]); // swap up since we outrank the
parent
            i = (i - 1) / 2; // move up and keep checking
        } // ends while statement
    } // ends siftUp function

    void siftDown(int i) { // creates the siftDown function that pushes an
email down until the heap is happy, 0(log n) for the same reason as siftUp
        int size = heap.size(), maxIdx = i, left = 2 * i + 1, right = 2 * i +
2; // instantiates the current size and both child indices
        if (left < size && heap[maxIdx] < heap[left]) maxIdx = left; // checks

```

```

if the left child outranks the current largest
    if (right < size && heap[maxIdx] < heap[right]) maxIdx = right; //
checks if the right child outranks the current largest
    if (maxIdx != i) { // checks if a child actually won
        swap(heap[i], heap[maxIdx]); // swaps down to that child
        siftDown(maxIdx); // recurses down to keep fixing the heap from
there, which also means  $O(\log n)$  extra stack space since its recursive
    } // ends if statement
} // ends siftDown function

public: // everything below here is public
    void push(const Email& email) { heap.push_back(email); siftUp(heap.size()
- 1); } // creates the push function that adds an email to the end then sifts
it up,  $O(\log n)$  overall

    void pop() { // creates the pop function that removes the top email from
the heap,  $O(\log n)$  overall because of the siftDown call
        if (heap.empty()) return; // checks if the heap is empty, if so theres
nothing to do
        heap[0] = heap.back(); // moves the last email into the root
        heap.pop_back(); // shrinks the vector by one
        if (!heap.empty()) siftDown(0); // fixes the heap from the root down
if theres anything left
    } // ends pop function

    const Email& top() const { return heap[0]; } // creates the top function
that looks at the top email without removing it,  $O(1)$  since its always at the
root

    bool empty() const { return heap.empty(); } // creates the empty function
that checks if the heap has nothing in it,  $O(1)$ 

    int size() const { return heap.size(); } // creates the size function that
returns how many emails are in the heap,  $O(1)$ 
}; // ends MaxHeap class

class CEOEmailManager { // creates the class CEOEmailManager that wraps the
heap and handles reading commands
    MaxHeap inbox; // instantiates inbox as the heap that holds all the CEO's
emails

public: // everything below here is public
    void processCommands(istream& input) { // creates the processCommands
function that reads commands from a stream and dispatches them,  $O(m \log n)$ 
overall for m lines since EMAIL and READ are each  $O(\log n)$ 
        string line; // instantiates line to hold each line as we read it
        while (getline(input, line)) { // reads lines until the input runs out
            if (!line.empty() && line.back() == '\r') line.pop_back(); //

```

```

strips a trailing carriage return if theres one
    if (line.empty()) continue; // skips blank lines

    if (line.rfind("EMAIL ", 0) == 0) { // checks if the line starts
with the EMAIL keyword
        stringstream ss(line.substr(6)); // instantiates a stream out
of everything after "EMAIL "
        string sender, subject, date; // instantiates the three fields
to be parsed into
        getline(ss, sender, ','); // grabs everything up to the first
comma as the sender
        getline(ss, subject, ','); // grabs everything up to the
second comma as the subject
        getline(ss, date, ','); // grabs whatever is left as the date
        inbox.push({sender, subject, date}); // creates an Email from
the three fields and pushes it onto the heap
    } else if (line == "NEXT") { // checks if the line is the NEXT
command
        if (!inbox.empty()) { // checks if theres actually something
to show
            const Email& e = inbox.top(); // grabs the top email
without removing it
            cout << "Next email:\n\tSender: " << e.sender
                << "\n\tSubject: " << e.subject
                << "\n\tDate: " << e.date << "\n\n"; // prints out
the sender, subject, and date
        } // ends if statement
    } else if (line == "READ") { // checks if the line is the READ
command
        inbox.pop(); // pops the top email off the heap
    } else if (line == "COUNT") { // checks if the line is the COUNT
command
        cout << "There are " << inbox.size() << " emails to
read.\n\n"; // prints how many emails are left
    } // ends else if statement
} // ends while statement
} // ends processCommands function
}; // ends CEOEmailManager class

int main() { // instantiates the main function for running the program
    CEOEmailManager manager; // creates the manager that holds the heap and
does all the work
    manager.processCommands(cin); // hands it stdin to read commands from
    return 0; // returns 0 for the main
} // ends main function

```

